

Assignment Summary

Text-to-SQL Agent using LangChain and OpenRouter

1. Overview

This project implements a Text-to-SQL agent that allows users to query a PostgreSQL database using plain English. The system translates natural language questions into valid SQL queries, executes them against a relational database, and returns the results in human-readable form. The agent is built on the LangChain framework and uses an LLM provided through the OpenRouter API.

2. Technology Stack

Component	Technology / Library
LLM Provider	OpenRouter API (via langchain-openrouter)
Agent Framework	LangChain (create_agent, tools)
Database	PostgreSQL
DB Connector	SQLAlchemy (create_engine, text)
Web Framework	FastAPI
Config Management	python-dotenv, Pydantic settings

3. Project File Structure

main.py

FastAPI application entry point. Defines the root HTTP endpoint.

agent.py

Initializes the LangChain agent with the LLM, tools, and system prompt. Invokes the agent on a user query.

tools.py

Defines the query_customer_db LangChain tool. Calls the LLM to generate SQL, executes it, and returns results.

prompts.py

Contains the system prompt that instructs the LLM to act as a read-only SQL assistant with strict safety rules.

llm.py

Configures the LLM instance using ChatOpenRouter with zero temperature for deterministic SQL generation.

database.py

Sets up the SQLAlchemy engine using PostgreSQL connection settings from environment variables.

4. How It Works

The system follows a straightforward pipeline where each step is handled by a dedicated module. Below is a description of the end-to-end flow.

Step 1 - User Input

The user provides a natural language question, for example: "List all customer names from USA." This is passed into the agent via the messages interface.

Step 2 - Agent Processing

The LangChain agent receives the message. Based on the question, it selects the appropriate tool, which in this case is `query_customer_db`. The agent is initialized with the LLM and the system prompt that defines rules and the database schema.

Step 3 - SQL Generation

Inside the tool, the question is sent to the LLM (via OpenRouter API) along with the system prompt. The LLM generates a valid PostgreSQL SELECT query. For example: `SELECT "customerName" FROM customers WHERE "country" = 'USA' LIMIT 100;`

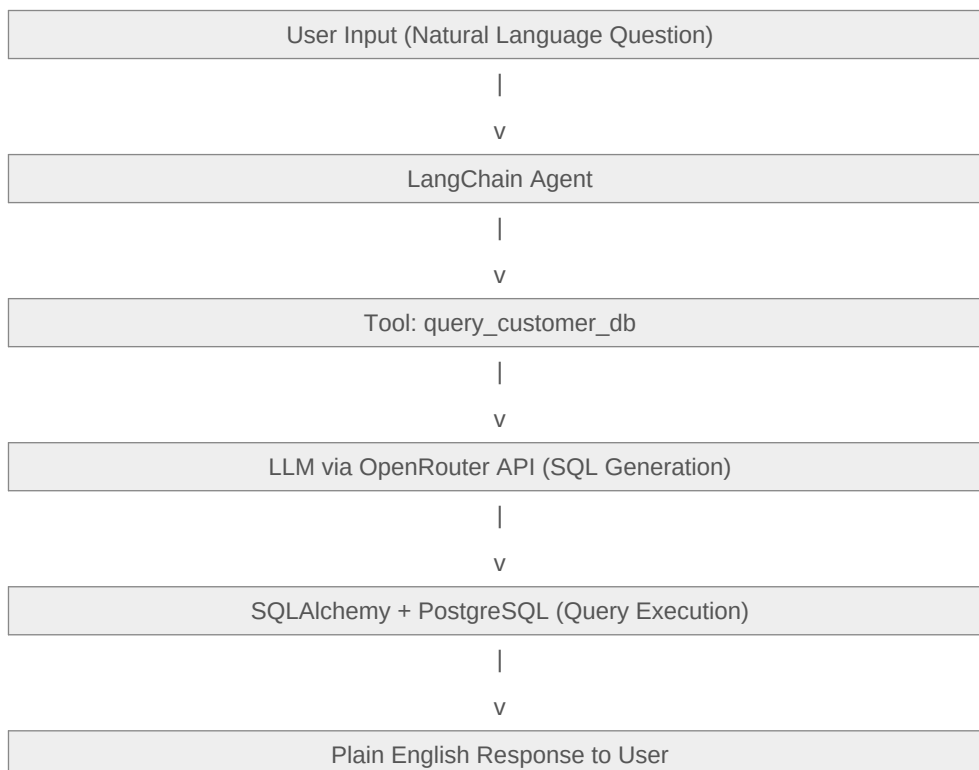
Step 4 - Query Execution

The generated SQL is executed against the PostgreSQL database using SQLAlchemy. The tool fetches all matching rows and returns the result count and data.

Step 5 - Response

The agent receives the tool output and formulates a plain English response for the user, without exposing any raw SQL or sensitive data columns.

5. Workflow Diagram



6. System Prompt and Safety Rules

A detailed system prompt is passed to the LLM at every invocation. It enforces the following rules:

- Only SELECT statements are allowed. No INSERT, UPDATE, DELETE, or DDL operations.
- Sensitive columns such as phone, email, addressLine1, postalCode are never exposed.
- All column and table names must be wrapped in double quotes (PostgreSQL convention).
- A LIMIT of 100 is applied to all non-aggregate queries by default.
- SELECT * is never used. Only necessary columns are selected.
- The LLM never returns raw SQL to the user. Output is always plain English.
- The full database schema is included in the prompt so the LLM can generate accurate queries.

7. Database Schema (Summary)

Table	Key Columns
customers	customerNumber, customerName, city, country, creditLimit
orders	orderNumber, orderDate, status, customerNumber
orderdetails	orderNumber, productCode, quantityOrdered, priceEach
products	productCode, productName, productLine, buyPrice, MSRP
productlines	productLine, textDescription
employees	employeeNumber, firstName, lastName, jobTitle, officeCode
offices	officeCode, city, country, territory
payments	customerNumber, checkNumber, paymentDate, amount

End of Assignment Summary